

Exercise 1: Configuring a Basic Spring Application

Scenario:

Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.

Steps:

1. Set Up a Spring Project:

- Create a Maven project named **LibraryManagement**.

`LibraryManagement`

- Add Spring Core dependencies in the **pom.xml** file.

2. Configure the Application Context:

- Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.

```
package com.library.repository;
```

```
public class BookRepository {
```

```
    public void displayRepository() {
```

```
        System.out.println("BookRepository Bean Created Successfully");
```

```
    }
```

```
}
```

- Define beans for **BookService** and **BookRepository** in the XML file.

```
package com.library.service;
```

```
public class BookService {
```

```
    public void displayService() {
```

```
        System.out.println("BookService Bean Created Successfully");
```

```
    }
```

```
}
```

3. Define Service and Repository Classes:

- Create a package **com.library.service** and add a class **BookService**.
- Create a package **com.library.repository** and add a class **BookRepository**

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository"/>

    <bean id="bookService" class="com.library.service.BookService"/>

</beans>
```

4. Run the Application:

- Create a main class to load the Spring context and test the configuration.

```
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.repository.BookRepository;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =

            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service = context.getBean("bookService", BookService.class);

        BookRepository repository = context.getBean("bookRepository",
        BookRepository.class);

        service.displayService();
    }
}
```

```
        repository.displayRepository();
    }
}
```

Exercise 2: Implementing Dependency Injection

Scenario:

In the library management application, you need to manage the dependencies between the `BookService` and `BookRepository` classes using Spring's IoC and DI.

Steps:

1. Modify the XML Configuration:

- Update **applicationContext.xml** to wire **BookRepository** into **BookService**.

```
package com.library.repository;

public class BookRepository {

    public void displayRepository() {

        System.out.println("Book Repository is Working...");

    }

}
```

BookService.java :

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {

        this.bookRepository = bookRepository;

    }

    public void displayService() {
```

```

        System.out.println("Book Service is Working...");

        bookRepository.displayRepository();
    }
}

```

2. Update the BookService Class:

- Ensure that **BookService** class has a setter method for **BookRepository**.

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService">
        <property name="bookRepository"
                  ref="bookRepository"/>
    </bean>
</beans>

```

3. Test the Configuration:

- Run the **LibraryManagementApplication** main class to verify the dependency injection.

```
package com.library;
```

```

import org.springframework.context.ApplicationContext;

import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =new
        ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =context.getBean("bookService", BookService.class);
        service.displayService();

    }

}

```

Exercise 3: Implementing Logging with Spring AOP

Scenario:

The library management application requires logging capabilities to track method execution times.

Steps:

1. Add Spring AOP Dependency:

- Update **pom.xml** to include Spring AOP dependency.

2. Create an Aspect for Logging:

- Create a package **com.library.aspect** and add a class **LoggingAspect** with a method to log execution times.

```

package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {

        this.bookRepository = bookRepository;
    }
}

```

```

    }

    public void displayService() {

        System.out.println("Book Service Method Executed");

        bookRepository.displayRepository();

    }

}

package com.library.aspect;

import org.aspectj.lang.ProceedingJoinPoint;

import org.aspectj.lang.annotation.Around;

public class LoggingAspect {

    @Around("execution(* com.library.service.*(..))")

    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws
        Throwable {

        long start = System.currentTimeMillis();

        Object object = joinPoint.proceed();

        long end = System.currentTimeMillis();

        System.out.println(joinPoint.getSignature().getName()+ " executed in " +
            (end - start)+ " ms");

        return object;

    }

}

```

3. Enable AspectJ Support:

- Update **applicationContext.xml** to enable **AspectJ** support and register the aspect.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/aop
           http://www.springframework.org/schema/aop/spring-aop.xsd">
    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>
    <bean id="bookService"
          class="com.library.service.BookService">
        <property name="bookRepository"
                  ref="bookRepository"/>
    </bean>
    <bean id="loggingAspect"
          class="com.library.aspect.LoggingAspect"/>
    <aop:aspectj-autoproxy/>
</beans>

```

4. Test the Aspect:

- Run the **LibraryManagementApplication** main class and observe the console for log messages indicating method execution times.

```

package com.library;

import org.springframework.context.ApplicationContext;

```

```
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =new
        ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service = context.getBean("bookService", BookService.class);

        service.displayService();

    }

}
```

Exercise 4: Creating and Configuring a Maven Project

Scenario:

You need to set up a new Maven project for the library management application and add Spring dependencies.

Steps:

1. Create a New Maven Project:

- Create a new Maven project named **LibraryManagement**.

2. Add Spring Dependencies in pom.xml:

- Include dependencies for Spring Context, Spring AOP, and Spring WebMVC.

3. Configure Maven Plugins:

- Configure the Maven Compiler Plugin for Java version 1.8 in the pom.xml file.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
```



```
<modelVersion>4.0.0</modelVersion>

<groupId>com.library</groupId>

<artifactId>LibraryManagement</artifactId>

<version>1.0-SNAPSHOT</version>

<packaging>jar</packaging>

<properties>

    <maven.compiler.source>1.8</maven.compiler.source>

    <maven.compiler.target>1.8</maven.compiler.target>

</properties>

<dependencies>

    <!-- Spring Context -->

    <dependency>

        <groupId>org.springframework</groupId>

        <artifactId>spring-context</artifactId>

        <version>5.3.30</version>

    </dependency>

    <!-- Spring AOP -->

    <dependency>

        <groupId>org.springframework</groupId>

        <artifactId>spring-aop</artifactId>

        <version>5.3.30</version>

    </dependency>

    <!-- Spring Web MVC -->

    <dependency>

        <groupId>org.springframework</groupId>
```

```
        <artifactId>spring-webmvc</artifactId>

        <version>5.3.30</version>

    </dependency>

    <!-- AspectJ Weaver -->

    <dependency>

        <groupId>org.aspectj</groupId>

        <artifactId>aspectjweaver</artifactId>

        <version>1.9.21</version>

    </dependency>

</dependencies>

<build>

    <plugins>

        <plugin>

            <groupId>org.apache.maven.plugins</groupId>

            <artifactId>maven-compiler-plugin</artifactId>

            <version>3.11.0</version>

            <configuration>

                <source>1.8</source>

                <target>1.8</target>

            </configuration>

        </plugin>

    </plugins>

</build>

</project>
```

Exercise 5: Configuring the Spring IoC Container

Scenario:

The library management application requires a central configuration for beans and dependencies.

Steps:

1. Create Spring Configuration File:

- Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
- Define beans for **BookService** and **BookRepository** in the XML file.

```
package com.library.repository;

public class BookRepository {

    public void displayRepository() {

        System.out.println("Book Repository Bean is Created");

    }

}

package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {

        this.bookRepository = bookRepository;

    }

    public void displayService() {

        System.out.println("Book Service Bean is Created");

        bookRepository.displayRepository();

    }

}
```

```
}
```

2. Update the BookService Class:

- Ensure that the **BookService** class has a setter method for **BookRepository**.

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository"/>

    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository"/>
    </bean>

</beans>
```

3. Run the Application:

- Create a main class to load the Spring context and test the configuration.

```
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {
```

```

        ApplicationContext context = new
        ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service= context.getBean("bookService", BookService.class);
        service.displayService();

    }

}

```

Exercise 6: Configuring Beans with Annotations

Scenario:

You need to simplify the configuration of beans in the library management application using annotations.

Steps:

1. Enable Component Scanning:

- Update **applicationContext.xml** to include component scanning for the **com.library** package.

```

package com.library.repository;

import org.springframework.stereotype.Repository;

@Repository

public class BookRepository {

    public void displayRepository() {

        System.out.println("Book Repository Bean Created");

    }

}

```

2. Annotate Classes:

- Use **@Service** annotation for the **BookService** class.
- Use **@Repository** annotation for the **BookRepository** class.

```

package com.library.service;

```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Service;
```

```
import com.library.repository.BookRepository;
```

```
@Service
```

```
public class BookService {
```

```
    @Autowired
```

```
    private BookRepository bookRepository;
```

```
    public void displayService() {
```

```
        System.out.println("Book Service Bean Created");
```

```
        bookRepository.displayRepository();
```

```
    }
```

```
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xmlns:context="http://www.springframework.org/schema/context"
```

```
    xsi:schemaLocation="
```

```
        http://www.springframework.org/schema/beans
```

```
        http://www.springframework.org/schema/beans/spring-beans.xsd
```

```
        http://www.springframework.org/schema/context
```

```
        http://www.springframework.org/schema/context/spring-context.xsd">
```

```
    <context:component-scan base-package="com.library"/>
```

```
</beans>
```

3. Test the Configuration:

- Run the **LibraryManagementApplication** main class to verify the annotation-based configuration.

```
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =new
        ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =context.getBean(BookService.class);
        service.displayService();

    }

}
```

Exercise 7: Implementing Constructor and Setter Injection

Scenario:

The library management application requires both constructor and setter injection for better control over bean initialization.

Steps:

1. Configure Constructor Injection:

- Update **applicationContext.xml** to configure constructor injection for **BookService**.

```
package com.library.repository;

public class BookRepository {
```

```

        public void displayRepository() {

            System.out.println("Book Repository Bean Created");

        }

    }
}

```

2. Configure Setter Injection:

- Ensure that the **BookService** class has a setter method for **BookRepository** and configure it in **applicationContext.xml**.

```

package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {

        this.bookRepository = bookRepository;

        System.out.println("Constructor Injection Performed");

    }

    public void setBookRepository(BookRepository bookRepository) {

        this.bookRepository = bookRepository;

        System.out.println("Setter Injection Performed");

    }

    public void displayService() {

        System.out.println("Book Service Bean Created");

        bookRepository.displayRepository();

    }

}

<?xml version="1.0" encoding="UTF-8"?>

```



```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="bookRepository" class="com.library.repository.BookRepository"/>
    <bean id="bookService"
          class="com.library.service.BookService">
        <constructor-arg ref="bookRepository"/>
        <property name="bookRepository" ref="bookRepository"/>
    </bean>
</beans>

```

3. Test the Injection:

- Run the **LibraryManagementApplication** main class to verify both constructor and setter injection.

```

package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =new
        ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =context.getBean("bookService", BookService.class);

        service.displayService();}}

```

Exercise 8: Implementing Basic AOP with Spring

Scenario:

The library management application requires basic AOP functionality to separate cross-cutting concerns like logging and transaction management.

Steps:

1. Define an Aspect:

- Create a package **com.library.aspect** and add a class **LoggingAspect**.

```
package com.library.repository;

public class BookRepository {

    public void displayRepository() {

        System.out.println("Book Repository Method Executed");

    }

}
```

2. Create Advice Methods:

- Define advice methods in **LoggingAspect** for logging before and after method execution.

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {

        this.bookRepository = bookRepository;

    }

    public void displayService() {

        System.out.println("Book Service Method Executed");

        bookRepository.displayRepository(); }}

package com.library.aspect;
```

```

import org.aspectj.lang.JoinPoint;

import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;

@Aspect

public class LoggingAspect {

    @Before("execution(* com.library.service.*(..))")

    public void beforeMethod(JoinPoint joinPoint) {

        System.out.println("Before Executing : " +
            joinPoint.getSignature().getName());

    }

    @After("execution(* com.library.service.*(..))")

    public void afterMethod(JoinPoint joinPoint) {

        System.out.println("After Executing : " +
            joinPoint.getSignature().getName());

    }

}

```

3. Configure the Aspect:

- Update **applicationContext.xml** to register the aspect and enable **AspectJ** auto-proxying.

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"

    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

    xmlns:aop="http://www.springframework.org/schema/aop"

    xsi:schemaLocation="

```

```

http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
<bean id="bookRepository" class="com.library.repository.BookRepository"/>
<bean id="bookService"
    class="com.library.service.BookService">
    <property name="bookRepository" ref="bookRepository"/>
</bean>
<bean id="loggingAspect" class="com.library.aspect.LoggingAspect"/>
<aop:aspectj-autoproxy/>
</beans>

```

4. Test the Aspect:

- Run the **LibraryManagementApplication** main class to verify the AOP functionality.

```

package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =new
        ClassPathXmlApplicationContext("applicationContext.xml");

        BookService service =context.getBean("bookService", BookService.class);

        service.displayService();

    }

```

}

Exercise 9: Creating a Spring Boot Application

Scenario:

You need to create a Spring Boot application for the library management system to simplify configuration and deployment.

Steps:

1. Create a Spring Boot Project:

- Use **Spring Initializr** to create a new Spring Boot project named **LibraryManagement**.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
    </parent>
    <groupId>com.library</groupId>
    <artifactId>LibraryManagement</artifactId>
    <version>1.0</version>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
</project>
```

2. Add Dependencies:

- Include dependencies for **Spring Web, Spring Data JPA, and H2 Database**.

```
spring.datasource.url=jdbc:h2:mem:librarydb
```

```
spring.datasource.driverClassName=org.h2.Driver
```

```
spring.datasource.username=sa
```

```
spring.datasource.password=
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

```
spring.h2.console.enabled=true
```

```
spring.h2.console.path=/h2-console
```

3. Create Application Properties:

- Configure database connection properties in **application.properties**.

```
package com.library.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity

public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    private String author;

    public Book() {

    }

    public Book(Long id, String title, String author) {

        this.id = id;

        this.title = title;

        this.author = author;

    }

    public Long getId() {

        return id;

    }

    public void setId(Long id) {

        this.id = id;

    }

}
```

```

    }

    public String getTitle() {

        return title;

    }

    public void setTitle(String title) {

        this.title = title;

    }

    public String getAuthor() {

        return author;

    }


    public void setAuthor(String author) {

        this.author = author;

    }

}

```

4. Define Entities and Repositories:

- Create **Book** entity and **BookRepository** interface.

```

package com.library.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.library.entity.Book;

public interface bookRepository extends JpaRepository<Book, Long> {

}

```

5. Create a REST Controller:

- Create a **BookController** class to handle CRUD operations.


```
package com.library.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.library.entity.Book;
import com.library.repository.BookRepository;

@RestController
@RequestMapping("/books")

public class BookController {

    @Autowired
    private BookRepository repository;

    @GetMapping
    public List<Book> getAllBooks() {
        return repository.findAll();
    }

    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return repository.save(book);
    }

    @PutMapping("/{id}")
    public Book updateBook(@PathVariable Long id, @RequestBody Book book) {
        book.setId(id);
        return repository.save(book);
    }

    @DeleteMapping("/{id}")
```

```
        public String deleteBook(@PathVariable Long id) {  
            repository.deleteById(id);  
            return "Book Deleted Successfully";  
        }  
    }  
}
```

6. Run the Application:

- Run the Spring Boot application and test the REST endpoints.

```
package com.library;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
  
public class LibraryManagementApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(LibraryManagementApplication.class,args);  
    }  
}
```